Interview Defense Dossier

Autonomous Perception & Planning for ADAS — YOLO + BEV + VLM on nuScenes

Project defense & interview preparation

Contents

How to use this document 1

PART I — THE PROJECT 2

1. Elevator pitches 2

2. The problem 2

3. The solution at a glance 2

4. System architecture 2

5. Component deep-dives 3
- 5.1 YOLOv8 obstacle detector . . . 3
- 5.2 BEV multi-modal trajectory planner . . . 3
- 5.3 Collision-risk fusion (`collision_risk.py`, pure NumPy) . . . 3
- 5.4 VLM reasoning layer (`vlm_reasoner.py`, neuro-symbolic) . . . 4

6. Data pipeline & AWS SageMaker 4

7. Business value 4

8. Key engineering decisions & trade-offs (your defense points) 5

9. Known limitations (own them honestly) 5

10. Metrics & facts cheat-sheet 5

PART II — MOCK INTERVIEW QUESTIONS 5
- 11. Easy . . . 6
- 12. Medium . . . 6
- 13. Hard . . . 7
- 14. “Tell me about this project” (STAR) . . . 8

15. Glossary 8

How to use this document

This is your single source of truth for defending the project in interviews. Part I is the narrative and technical reference; Part II is a bank of mock questions (easy / medium / hard) with model answers. Read Part I once end-to-end, then drill Part II. Numbers worth memorising are collected in the cheat-sheet (§10).

PART I — THE PROJECT

1. Elevator pitches

15-second: "I built an end-to-end autonomous-driving perception and planning stack on the nuScenes dataset — camera object detection plus a LiDAR bird's-eye-view trajectory planner — and fused them into a real collision-risk decision (CLEAR / CAUTION / BRAKE) with a local vision-language model that explains the scene in natural language."

60-second: "Most driver-assistance demos stop at drawing boxes. Mine goes the whole loop: perceive, predict, decide. A YOLOv8 detector finds road obstacles in the camera feed; a custom convolutional network turns the LiDAR point cloud into a 500×500 bird's-eye view and predicts several possible future trajectories for the ego vehicle. A pure-NumPy fusion layer walks the predicted path through the LiDAR obstacle map and outputs distance-to-obstacle, time-to-obstacle, and a CLEAR/CAUTION/BRAKE decision. On top, a local VLM — Qwen via Ollama — reasons over the full six-camera surround plus the BEV to flag open-vocabulary hazards a detector can't name and to give the driver a spoken instruction. It was trained on AWS SageMaker against the 200 GB dataset, and it runs locally as a Streamlit app with a video monitor and a per-scene PDF safety report."

2-minute (add): the business framing — the same module feeds ADAS warn/brake, fleet/insurance risk scoring, and AV validation; the engineering story — I recovered a lost model architecture from its weights, diagnosed a straight-line mode-collapse failure, and kept everything honest and reproducible; and the neuro-symbolic angle — geometry for precise measurement, language for meaning.

2. The problem

Forward-collision warning (FCW) and automatic emergency braking (AEB) are mandated or incentivised by safety regulators worldwide (Euro NCAP, NHTSA). The underlying capability is a closed loop: **perceive the scene → predict the near-future motion → flag any conflict.** Building it is expensive because it spans perception (what's around me) and planning/prediction (where am I going), usually owned by separate teams. The project demonstrates one engineer owning that full vertical slice on a real autonomous-driving dataset.

3. The solution at a glance

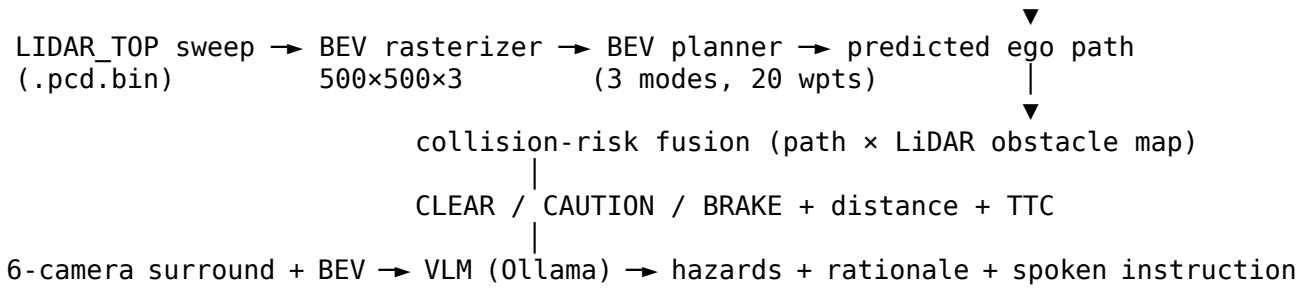
A two-sensor, multi-stage pipeline:

1. **Perception (camera):** YOLOv8 detects obstacles; an optional COCO-pretrained variant names them.
2. **Planning (LiDAR):** a BEV CNN predicts 3 candidate ego trajectories (20 waypoints each) with mode probabilities and per-waypoint confidence.
3. **Decision (fusion):** the predicted path is tested against the LiDAR obstacle map → distance, TTC, and CLEAR/CAUTION/BRAKE.
4. **Reasoning (VLM):** a local vision-language model reasons over the 6-camera surround + BEV to add open-vocabulary hazards, corroborate the decision, and produce a spoken driver instruction.

Delivered as a Streamlit app (4 tabs), a risk-monitor video generator, a per-scene PDF safety report, and a narrated multi-scene demo.

4. System architecture

```
CAM_FRONT image ─► YOLOv8 detector ─► obstacle boxes (named) ─┐
```



5. Component deep-dives

5.1 YOLOv8 obstacle detector

- Fine-tuned YOLOv8-nano on **20,256** nuScenes camera frames, image size 640, 20 epochs, batch 16, on **CPU** (compute-limited).
- Trained as a **single class** (`object`) so the warning logic is trivial: any confident detection is a hazard, and it halves the labelling/learning burden on a CPU budget.
- **Named detection without retraining:** a stock COCO-pretrained YOLOv8 filtered to road classes (ids 0,1,2,3,5,7 = person, bicycle, car, motorcycle, bus, truck) provides typed detections for free.
- **Metrics (epoch 20):** Precision 0.64, Recall 0.30, mAP@0.50 0.35, mAP@0.50–0.95 0.21. mAP rose monotonically 0.04→0.35 across all 20 epochs — undertrained, not converged.

5.2 BEV multi-modal trajectory planner

- **Input:** 500×500×3 BEV raster from the LiDAR cloud (channels = height / intensity / density), ROI ±50 m, 0.2 m/cell.
- **Encoder:** VGG-style — stem (Conv7×7 s2 → BN → ReLU → MaxPool) then three double-conv stages, 64→128→256→512 channels.
- **Heads (shared 256-d embedding):** 3 trajectory regressors (each 20 waypoints × xy = 40 outputs), a mode classifier (softmax over 3), a per-waypoint confidence head, and an auxiliary 8-class BEV segmentation decoder.
- **Why multi-modal:** at a junction the "correct" future isn't unique; collapsing to one path hides risk, so we predict several weighted hypotheses (standard in modern motion-forecasting).
- **Metrics (epoch 5 checkpoint):** val ADE 3.29 m, val FDE 7.23 m. Early checkpoint with a sensible "go straight" prior; the v3 training recipe (rotation/flip augmentation + mode-diversity loss) targets the straight-line collapse.
- **ADE/FDE:** ADE = mean L2 over all waypoints; FDE = L2 at the final waypoint.

5.3 Collision-risk fusion (`collision_risk.py`, pure NumPy)

- **Obstacle map:** a BEV cell is an obstacle if it is both *dense* (density > 0.2) and *elevated* (height > 0.55 normalised) — the height test removes flat road that a density-only map keeps.
- **Corridor scan:** walk the predicted path outward; at each waypoint test a ±1 m corridor (vehicle half-width) against the obstacle map, ignoring an ego deadzone (< 3 m, the vehicle's own returns). First hit = nearest in-path obstacle distance `d`.
- **Speed & TTC:** ego speed ≈ path length / horizon (horizon = 20 waypoints × 0.5 s keyframe spacing); `TTC = d / v`.
- **Decision:** `d < 8 m → BRAKE`, `8 ≤ d < 20 m → CAUTION`, else `CLEAR`.
- **Validation:** on a 120-frame scene the distribution is realistic (closest approaches 3.5–33 m), not all-clear or all-collision.

5.4 VLM reasoning layer (`vlm_reasoner.py`, neuro-symbolic)

- A local VLM (Qwen-VL family via **Ollama**) receives the **6-camera surround montage + the BEV render** as one composite image, plus a text summary of the geometric output (risk, distance, TTC, speed).
- Returns strict JSON: scene caption, **open-vocabulary hazards** (construction, emergency vehicles, traffic-light state, debris, animals, vulnerable road users, weather), agreement with the CLEAR/CAUTION/BRAKE decision + rationale, recommended action, and a **spoken driver instruction**.
- **Why surround + BEV, not just the front camera:** the driver already sees the front; the value is synthesising all views the human can't hold at once. The VLM can't read raw point clouds, so LiDAR is supplied as the rendered BEV image.
- **Pluggable backend:** vision models (`qwen3.5:cloud`, `qwen2.5vl`) see the images; text-only models (`qwen3:8b`, `phi3.5`) reason over the summary; a mock backend runs the pipeline with no GPU/server.
- **Output is spoken** via the browser Web Speech API (TTS), forced to en-US; the *constructed instruction* is read aloud, not the raw analysis.

6. Data pipeline & AWS SageMaker

Developed on an **AWS SageMaker** notebook instance against the full ~200 GB nuScenes release. Pipeline (`data_preparation_v2.py`):

1. **Parse metadata** — link each keyframe `sample` to its LIDAR_TOP sweep, ego pose, calibrated sensors, and 3-D annotations (nuScenes stores these as relational JSON tables).
2. **Build ground-truth ego trajectory** — read future ego poses, transform into the current ego frame (`p_ego = R⁻¹·(p_world − t)`), interpolate to 20 fixed waypoints.
3. **Rasterize LiDAR → BEV** — project points into a 500×500 top-down grid with height/intensity/density channels.
4. **Train** (`train_trajectory_v3.py`) — rotation (±25°) and horizontal-flip augmentation applied jointly to BEV + trajectory, plus a mode-diversity loss, to break the "always go straight" collapse.

Local inference is intentionally light: weights ship in the repo (Git LFS), and every tool runs on a laptop CPU against bundled sample frames. The original `trajectory_model.py` was lost with the SageMaker instance and **reconstructed from the checkpoint** — all 130 weight tensors matched exactly.

7. Business value

Market	Consumes	Output used
ADAS / OEM	real-time warn & brake	per-frame CLEAR/CAUTION/BRAKE + TTC
Fleet & usage-based insurance	per-drive risk score	risk timeline + near-miss events (PDF report)
AV research / data-ops	scene triage & labelling	multi-modal trajectories + scene risk profile

The decisive quantities — distance-to-obstacle and time-to-obstacle — are common to all three; the same pipeline serves each, only the consumer changes.

8. Key engineering decisions & trade-offs (your defense points)

- **Single-class YOLO + separate COCO model for names** — simplest warning logic on a CPU budget; named detection added with zero training instead of relabelling 20k frames.
- **LiDAR as a rendered BEV image for the VLM** — VLMs can't ingest point clouds; rendering the BEV is a pragmatic, legitimate bridge (and it's exactly what the geometric stack already produces).
- **Side-by-side dashboard, not LiDAR-painted-on-image** — true projection needs calibration matrices (intrinsics + extrinsics) not bundled here; the multi-panel view is the field standard absent calibration. A `project_lidar_to_image()` hook is stubbed for the upgrade.
- **Construct a driver instruction, then TTS it** — reading the raw analysis aloud is noise; an ADAS co-pilot speaks one imperative.
- **Pure-NumPy fusion** — unit-testable, no torch, validated directly on real frames.
- **Timestamp-based camera↔LiDAR sync** — nuScenes filenames carry microsecond timestamps; nearest-match within ±0.5 s, returns nothing rather than mispairing.
- **Recovering the model from weights** — extracted every parameter name+shape from the checkpoint and rebuilt the `nn.Module` to load with `strict=True`.

9. Known limitations (own them honestly)

- **Planner is an epoch-5 checkpoint** — predicts mostly straight paths; FDE 7.23 m. Fix: resume training with the v3 augmentation/diversity recipe.
- **Detector undertrained** — single class, CPU, 20 epochs, still improving. Fix: multi-class GPU training.
- **No cross-sensor geometric fusion** — risk uses the LiDAR obstacle map only; camera detections aren't projected into BEV (needs calibration).
- **VLM adds semantics, not geometric accuracy** — it's an explainability/coverage layer, and per-frame (not token-streaming) narration.
- **No real-time guarantees** — not latency-profiled or exported to ONNX/TensorRT.

These are *strengths in disguise* in an interview: they show you know what "production" would require.

10. Metrics & facts cheat-sheet

Item	Value
nuScenes raw dataset	~200 GB
YOLO training images	20,256
Bundled LiDAR frames / scenes	3,376 / 67
BEV raster	500 × 500 × 3 (±50 m, 0.2 m/cell)
Planner params / tensors	~12M / 130
Trajectory modes / horizon	3 / 20 waypoints (~ a few seconds)
YOLO P / R / mAP50 / mAP50-95	0.64 / 0.30 / 0.35 / 0.21
Planner val ADE / FDE	3.29 m / 7.23 m
Risk thresholds	BRAKE < 8 m, CAUTION 8–20 m
Ego deadzone / corridor	3 m / ±1 m

PART II — MOCK INTERVIEW QUESTIONS

Answers are below each question. Practice saying them out loud.

11. Easy

E1. What does this project do, in one sentence? > It detects road obstacles from camera, predicts the ego vehicle's near-future path from LiDAR, fuses the two into a CLEAR/CAUTION/BRAKE decision with distance and time-to-collision, and uses a vision-language model to explain the scene and voice an instruction to the driver.

E2. What is a bird's-eye-view (BEV) representation and why use it? > A top-down 2-D grid built by projecting the 3-D LiDAR point cloud onto the ground plane. It's a compact, ego-centric, metric representation that's natural for planning (distances and free space are directly readable) and easy for a CNN to process — here 500×500 cells over ±50 m, with height/intensity/density channels.

E3. What are ADE and FDE? > Average Displacement Error — the mean L2 distance between predicted and ground-truth waypoints over the whole trajectory. Final Displacement Error — the L2 distance at the last waypoint. They're the standard motion-forecasting metrics; mine are 3.29 m and 7.23 m.

E4. Why YOLO for detection? > It's a fast, single-stage detector with a strong speed/accuracy trade-off, which matters for a real-time ADAS context, and YOLOv8 has a mature, easy-to-fine-tune toolchain (Ultralytics).

E5. What is TTC and how do you compute it? > Time-to-collision: the nearest in-path obstacle distance divided by the estimated ego speed. Speed is estimated from the predicted path length over its time horizon. It's the core quantity an AEB system thresholds on.

E6. What does the VLM add that the detector doesn't? > Open-vocabulary understanding and explanation. YOLO only knows its trained classes; the VLM can flag a construction zone, an emergency vehicle, a red light, or debris, and it explains *why* the system is braking in natural language — an independent, auditable second channel.

12. Medium

M1. Why predict multiple trajectories instead of one? > Future motion is inherently multi-modal — at an intersection the car could go straight, turn left, or turn right, and a single averaged prediction is both inaccurate and unsafe (it can land between the real options). Predicting several weighted hypotheses lets a safety layer reason about the worst plausible case. My model outputs 3 modes with a softmax probability and per-waypoint confidence.

M2. How does the collision-risk fusion actually work? > I build an obstacle map from the BEV by keeping cells that are both dense and elevated above the road surface, which removes ground returns. Then I walk along the predicted path and, at each waypoint beyond a 3 m ego deadzone, test a ±1 m corridor against that map. The first hit gives the nearest in-path obstacle distance; dividing by estimated speed gives TTC; thresholds map that to CLEAR/CAUTION/BRAKE. It's pure NumPy so it's unit-testable.

M3. How do you synchronise the six cameras with the LiDAR frame? > By timestamp. nuScenes filenames embed a microsecond capture time, which is globally unique, so I pair each LiDAR frame to the camera frame with the nearest timestamp within ±0.5 s. If nothing is within the window it returns nothing rather than pairing a wrong-scene frame. I also split the LiDAR stream into scenes by detecting >2 s timestamp gaps.

M4. Why feed the VLM a rendered BEV image instead of the raw point cloud? > Vision-language models accept images, not point clouds. Rendering the LiDAR to a top-down BEV image — which the geometric pipeline already produces — lets the VLM reason about depth and free space alongside the cameras. It's a deliberate, pragmatic bridge; the honest caveat is it's not native point-cloud reasoning.

M5. How did you handle losing the model's source code? > Only the 146 MB checkpoint survived. I read the checkpoint's state-dict and extracted every parameter's name and tensor shape with a custom unpickler — no torch build needed — then inferred each layer from its key path and shapes and rebuilt the `nn.Module`. I verified it by a set-difference: zero missing, zero unexpected,

zero shape mismatches, so it loads with `strict=True`. The reconstructed heads also matched the original output visualisations.

M6. What's the business case — who would pay for this? > Three markets off one pipeline: OEM/ADAS teams need the real-time warn/brake trigger; fleet operators and usage-based insurers need a per-drive risk score and near-miss log, which is exactly my PDF safety report; and AV research/data-ops teams need scene triage and auto-labelling, which the multi-modal trajectories and scene risk profile provide. Distance-to-obstacle and TTC are the common currency.

M7. Why is the detector's mAP only 0.35 — isn't that bad? > It's undertrained, not broken. It was a single class trained for 20 epochs on CPU, and the mAP curve was still rising monotonically (0.04→0.35) with no plateau — it was compute-limited. The pipeline is correct; more epochs on GPU and multi-class labels would lift it. I deliberately report the real numbers rather than cherry-pick.

M8. How would you turn this into a real-time system? > Replace the per-point BEV rasterizer with a vectorised/GPU version, run the planner and detector on GPU, profile end-to-end latency, and export to ONNX/TensorRT for an embedded target. The fusion is already cheap NumPy. I'd also move the VLM off the hot path — it's an explanation/triage layer, not the millisecond brake loop.

13. Hard

H1. Your fusion calls something an obstacle if it's "dense and elevated." What are the failure modes, and how would you make it robust? > Failure modes: low obstacles below the height threshold (a curb, a child lying down) get missed; overhanging structures (bridges, foliage) trigger false positives; sparse-but-real distant objects fall under the density threshold; and there's no temporal consistency, so a one-frame LiDAR dropout flips the decision. Robustness: add ground-plane segmentation (e.g., RANSAC) instead of a fixed height cut, cluster points into objects rather than per-cell tests, track obstacles across frames with a Kalman filter for temporal smoothing, and calibrate thresholds against labelled near-miss data instead of hand-tuning. Ultimately I'd fuse the camera detections into the same BEV for redundancy.

H2. The VLM is non-deterministic and can hallucinate. How can you put a language model anywhere near a safety system? > You don't put it in the braking loop. The geometric stack (detector + planner + fusion) is the deterministic, low-latency safety path; the VLM is a *parallel* advisory and explanation layer — open-vocabulary hazard flags, an auditable rationale, and driver-facing language. I constrain it with a strict JSON schema and forced JSON decoding, keep temperature low, and treat its output as advisory corroboration, never as the actuator. For production you'd add guardrails: schema validation, confidence gating, and a rule that the VLM can only *raise* caution, never *lower* it below the geometric decision.

H3. Walk me through the coordinate transforms from a raw LiDAR point to a BEV cell and to a trajectory waypoint. > A LiDAR point is in the sensor frame (x forward, y left, z up). For the BEV I filter to the ROI (±50 m, $z \in [-3,3]$) and map to grid indices `gx = floor((x+50)/0.2)`, `gy = floor((y+50)/0.2)`, accumulating height/intensity/density per cell. For the ground-truth trajectory, future ego positions are in the global frame; I transform each into the *current* ego frame with `p_ego = R⁻¹·(p_world − t)` where `R, t` are the current ego rotation and translation, then interpolate to 20 waypoints. Predictions live in that same ego frame, so they overlay the BEV directly.

H4. Why multi-modal regression rather than, say, an occupancy/heatmap or a generative trajectory model? > Multi-mode regression with a mode classifier is a strong, simple baseline: it gives a small set of interpretable, weighted paths that a planner can directly consume, and it trains stably with a "best-of-K" regression loss plus a diversity term to avoid collapse. Heatmaps capture uncertainty richly but need a decoding step to get trajectories and can be heavy; generative models (CVAE, diffusion) give diverse samples but are harder to train and to certify. Given the scope and compute, fixed-K multi-modal regression was the right cost/benefit — and it's what several production-grade forecasters (e.g., MultiPath-style) use.

H5. Mode collapse — how do you know it happened, and how does the diversity loss fix it? > Symptom: the three predicted modes converge to nearly the same path (almost always "go straight"), because most driving is straight so the data is heavily imbalanced toward it and a plain

regression loss is happy to ignore turns. I see it directly in the visualisations — the three colored paths overlap. The v3 fixes are (a) augmentation: random rotation and horizontal flip applied jointly to the BEV and the trajectory, which manufactures turn examples, and (b) a diversity term in the loss that penalises modes for being too close to each other, forcing them to spread across plausible futures. Combined with a winner-take-all/best-mode regression loss, each mode specialises.

H6. If you had to certify the distance/TTC numbers for safety, what's missing? > A lot. There's no calibration of the LiDAR-to-vehicle extrinsics in my distance math (I assume the sensor origin is the ego origin); no sensor-noise model or confidence interval on the distance; no temporal filtering; the speed is *estimated* from the predicted path, not measured from odometry/IMU; and the thresholds are hand-set, not validated against labelled collisions. To certify I'd ground distance in calibrated geometry, fuse measured ego speed, add uncertainty propagation, validate thresholds on a labelled near-miss dataset with precision/recall on the BRAKE event, and run hardware-in-the-loop testing.

H7. A skeptic says "this is just gluing pretrained models together." Respond. > Two of the three core models are *mine*: the BEV planner is a custom multi-task network I trained from the raw nuScenes data, and the fusion/decision logic is original and unit-tested. I also did the harder systems work: the data pipeline from 200 GB of relational annotations to BEV tensors and ego trajectories, recovering a lost architecture from weights, diagnosing and fixing mode collapse, and turning model outputs into a product (risk video, safety report, spoken co-pilot). The pretrained pieces — YOLO for detection, the VLM for semantics — are used where buying beats building, which is itself an engineering judgment.

H8. What would you do differently if you started over? > Train the planner to convergence on GPU from day one with the augmentation recipe baked in, so I'd report real ADE/FDE near state-of-the-art rather than an early checkpoint. I'd keep the calibration metadata so I could do true sensor fusion and LiDAR-on-image projection. I'd version data and experiments (DVC + MLflow) instead of relying on a notebook instance — which is exactly why I lost the model source. And I'd define the safety metric (BRAKE precision/recall on labelled events) up front and optimise toward it.

14. "Tell me about this project" (STAR)

- **Situation:** I had a months-old, half-finished autonomous-driving project on SageMaker — untested models, lost source, no clear value story.
- **Task:** turn it into a coherent, defensible portfolio piece with real, demonstrable value.
- **Action:** recovered the BEV model from its weights; built a collision-risk fusion layer turning detections + trajectory into a CLEAR/CAUTION/BRAKE decision with TTC; added a local VLM reasoning layer over the surround + BEV with a spoken driver instruction; and packaged everything as a Streamlit app, a risk-monitor video, and a PDF safety report.
- **Result:** a runnable, end-to-end ADAS perception-planning-decision stack with an honest metrics story and a clear three-market business case — and a neuro-symbolic angle (geometry + language) that's current with where the field is going.

15. Glossary

- **ADAS** — Advanced Driver-Assistance System (FCW, AEB, lane-keep, etc.).
- **BEV** — Bird's-Eye View; top-down grid representation.
- **ADE / FDE** — Average / Final Displacement Error (trajectory metrics).
- **TTC** — Time-To-Collision.
- **mAP** — mean Average Precision (detection metric); @0.50 and @0.50–0.95 are IoU thresholds.
- **Mode collapse** — multiple predicted trajectories degenerating to one.
- **VLM** — Vision-Language Model.
- **Neuro-symbolic** — combining learned/geometric components with language reasoning.
- **nuScenes** — large public autonomous-driving dataset (camera, LiDAR, radar, annotations).